PRACTICAL - 10

Aim: Thread-pool runner (TaskRunner.java + Driver.java)

Code Screenshot:

TaskRunner.java

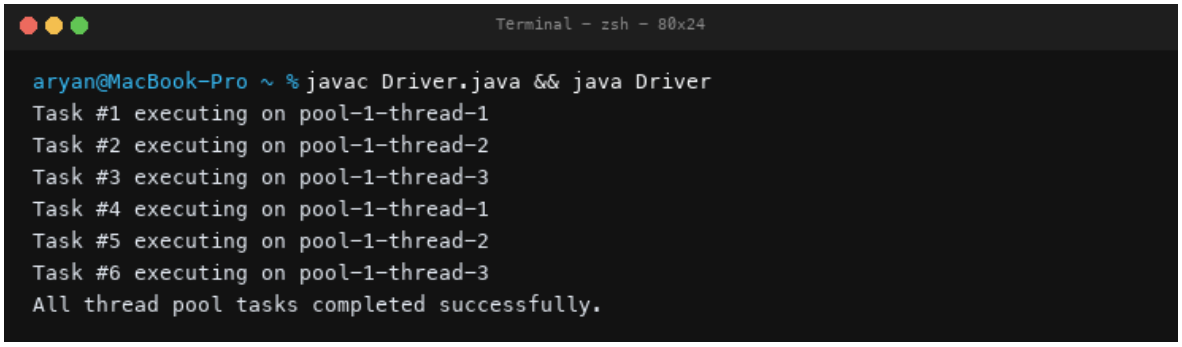
```java
public class TaskRunner implements Runnable {
    private int id;
    public TaskRunner(int id) { this.id = id; }
    public void run() {
        String thread = Thread.currentThread().getName();
        System.out.println("Task #" + id + " executing on " + thread);
        try { Thread.sleep(50); } catch (InterruptedException e) {}
    }
}
```

Driver.java

```java
import java.util.concurrent.*;

public class Driver {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);
        for (int i = 1; i <= 6; i++) {
            pool.submit(new TaskRunner(i));
        }
        pool.shutdown();
        pool.awaitTermination(5, TimeUnit.SECONDS);
        System.out.println("All thread pool tasks completed successfully");
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java concurrency with ExecutorService and fixed thread pools.
- Observed thread recycling across queued asynchronous tasks.
- Managed lifecycle termination using shutdown() and awaitTermination().
- Avoided excessive thread allocation overhead in high-throughput workloads.
- Learned how worker queues decouple task submission from execution.

Aim: Producer–consumer (SharedBuffer.java + Driver.java)

Code Screenshot:

SharedBuffer.java

```java
public class SharedBuffer {
    private int data;
    private boolean hasData = false;

    public synchronized void produce(int val) throws InterruptedException {
        while (hasData) wait();
        data = val;
        hasData = true;
        System.out.println("Produced: " + val);
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (!hasData) wait();
        hasData = false;
        System.out.println("Consumed: " + data);
        notify();
        return data;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        SharedBuffer buffer = new SharedBuffer();
        Thread prod = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) buffer.produce(i * 10);
            } catch (InterruptedException e) {}
        });
        Thread cons = new Thread(() -> {
            try {
                for (int i = 1; i <= 3; i++) buffer.consume();
            } catch (InterruptedException e) {}
        });
        prod.start(); cons.start();
        prod.join(); cons.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered inter-thread signaling with wait() and notify().
- Implemented guarded wait loops (while (condition) wait()) to handle spurious wakeups.
- Solved the classic Producer-Consumer synchronization coordination problem.
- Guaranteed zero message loss across asynchronous producer and consumer threads.
- Gained foundational skills for building thread-safe communication pipelines.